Keyboard and Mouse Simulations
WebDriver - Lecture - 6

Agenda

- Methods for Dynamic Waiting Strategy
- Examples for Explicit Waiting
- Actions class in WebDriver
- “Rename Folder” Script in Box Portal
- Mouse Hover action
- Drag And Drop Mouse Action
- Keyboard Simulation for Copy-Paste
- Keys class in WebDriver
- Quiz
- Homework

Methods for Dynamic Waiting Strategy

- There are some newer **conditions** in **ExpectedConditions class** which can be used for better **waiting strategy** i.e. for extensive validations and adding synchronization points in test scripts:

- Attribute Validation: These methods will take **3** arguments (**By locator, some other attribute Name, attribute value**) and will **return true** if attribute has a given value within the timeout duration otherwise these methods will **throw Timeout Exception**
  - **wait.until(ExpectedConditions.attributeContains(By.cssSelector("box"),"name","txt"));**
  - **wait.until(ExpectedConditions.attributeToBe(By.cssSelector("box"),"name","txt1"));**

  Below method takes 2 arguments **(WebElement, attribute name)** and will return **true** if given attribute is **not null and has some value** within timeout duration
  - **wait.until(ExpectedConditions.attributeToBeNotEmpty(element,"name"));**

- Waiting for Elements Count: These method takes **2** arguments (**By locator, Elements Number**) and will **return List<WebElement>** if **Elements Number** is attained for given **By Locator** within the timeout duration otherwise these methods will **throw Timeout Exception**
  - **wait.until(ExpectedConditions.numberOfElementsToBe(By.cssSelector("box"),5));**
  - **wait.until(ExpectedConditions.numberOfElementsToBeLessThan(By.cssSelector("box"),5));**
  - **wait.until(ExpectedConditions.numberOfElementsToBeMoreThan(By.cssSelector("box"),5));**

- Waiting for a text in an element: These method takes **2** arguments (**By locator, element text**) and will **return true** if **inner HTML text** has a given value within the timeout duration otherwise these methods will **throw Timeout Exception**
  - **wait.until(ExpectedConditions.textMatches(By.cssSelector("box"), Pattern.compile("textStringPart")));**
  - **wait.until(ExpectedConditions.textToBe(By.cssSelector("box"),"textString"));**

Examples for Explicit Waiting

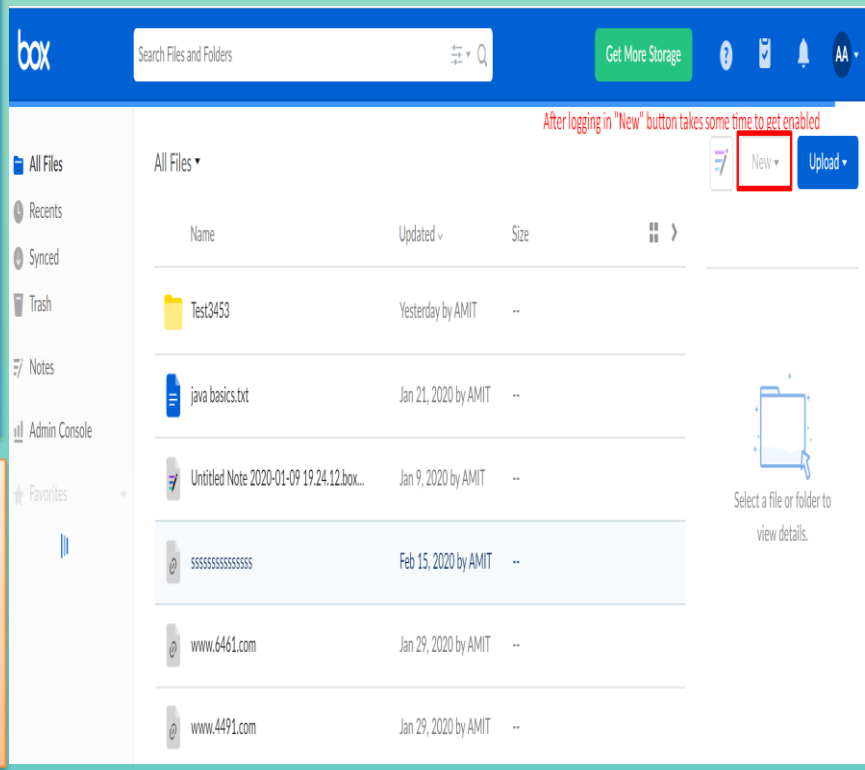
- In Box portal, "**New**" button takes some time to get **enabled**. Lets try with three options to make "**New**" button enabled using **dynamic waiting**:

- **Attribute Validation**: Before getting **WebElement** of "**New**" button, we can wait for the bar which contains this button HTML: *** ***

  boolean flag =wait.until(ExpectedConditions.*attributeToBe(* By.*cssSelector("div[class*='ItemListActionBar']"), "data-testid", "actionbar"));*

- **Elements Count:** Before getting **WebElement** of "New" button, we can wait for the element count of **"Try Box Notes"** button

  List<WebElement> list = wait.until(ExpectedConditions.*numberOfElementsToBeMoreThan(By.xpath("//button[@class='btn explore-button']"), 0));*



- **Text in an element:** Before getting **WebElement** of "**New**" button, we can also wait for the **inner HTML** text of the button which is 'New'

  boolean flag = wait.until(ExpectedConditions.*textToBe(By.xpath("//button[contains(@id,'menubutton')]//span[text()='New'] "), "New"));*

Actions class in WebDriver

- The **Actions class** is a **API** for emulating complex ***keyboard and mouse actions*** *in* ***Selenium Webdriver*** and is in the package '**org.openqa.selenium.interactions**'
- It contains the **Actions** class and the **Action** Interface that are needed when executing these events
- It Implements builder pattern, in which a complex object is constructed by passing **WebDriver** object in **Actions** class constructor
- You can build a series of actions (**Action Chaining**) of user gestures with Actions class methods
- **Action Chaining** is possible because all methods of **Actions** class returns the Actions class object except **build()** and **perform()** methods

- **build()** method of **Actions** class compiles all events and **returns object** of **Action interface**
- **perform()** method of **Action interface** executes all events
- So **build()**. **perform()** is used to compile and execute the **Actions class**

- **Some important methods:**

contextClick() //Performs a Right-click at the current mouse location

dragAndDrop(source, target) //drags the source element to destination

moveToElement(toElement) // Moves the mouse to middle of the element

keyDown(modifier_key) // Performs a modifier key press. Does not release the modifier key - subsequent interactions may assume it's already pressed.

Keyboard and Mouse Events

Mouse Events:

- clickAndHold()
- contextClick()
- doubleClick()
- dragAndDrop()
- dragAndDropBy()
- moveByOffset()
- moveToElement()
- release()

Keyboard Events:

- keyDown()
- keyUp()
- sendKeys()

Creates action class object

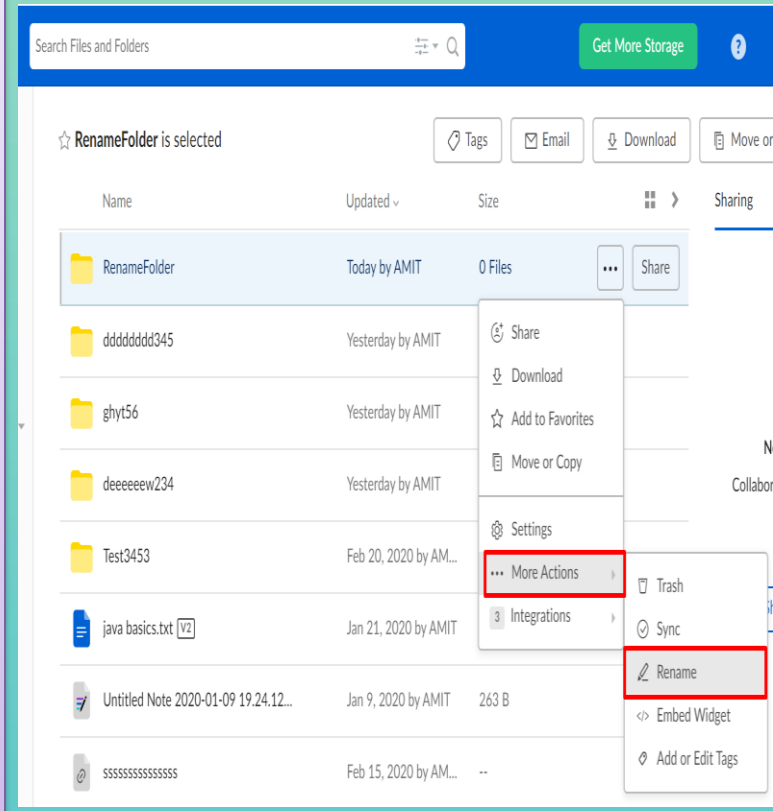
```
Actions action = new Actions(driver);
action.moveToElement(moreActionsElement).build().perform();
```

used for mouse hover

“Rename Folder” Script in Box Portal

- **WebDriver** script for Renaming Folder:
  1. Navigate to http://app.box.com
  2. Login into app
  3. Click on “**RenameFolder**” row to select it
  4. Click on “**More Options**” button in the folder row
  5. **Mouse Hover** “**More Actions**” element in the options list
  6. Click on “**Rename” link**
  7. Rename the folder to “**RenameFolder12345**” in the textbox
  8. Verify the “**Rename message**” notification and wait for it to disappear from the screen
  9. Mouse Hover “**RenameFolder12345**” name link, so that pencil icon is visible
  10. Click on the “**pencil icon”** link
  11. Rename the folder back to “**RenameFolder**” in the textbox
  12. Verify the “**Rename message**” notification and wait for it to disappear from the screen
  13. **Logout** from the portal

Note: You only have to Mouse hover on “**More Actions**” element and not to click on it



Mouse Hover Action

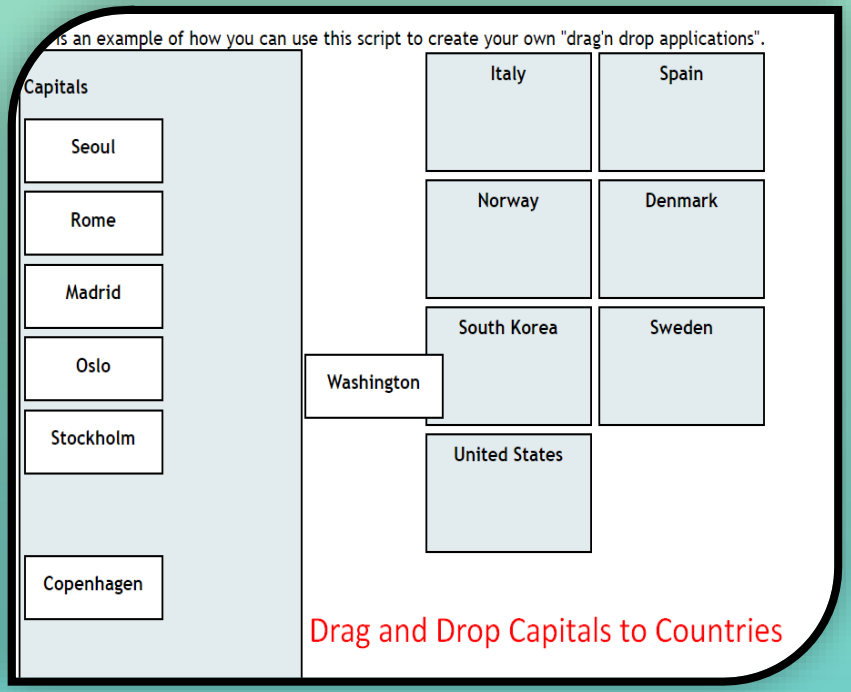
- Renaming the folder involves **mouse hover gesture**. **Steps** for renaming folder:
  - Select the folder row which has hyperlink tag with title attribute as "**RenameFolder**"
  - For this selected row, locate "**More Options**" element and click on it
  - In the Options list, locate "**More Actions**" element link and mouse hover on it:
    - **Actions moveToElement(WebElement target)** method – moves the mouse **cursor** over the target **WebElement**
    - This method returns **Actions class** object
- Click on "**Rename Folder**" link and rename the folder.

```
String testFolder = "RenameFolder";
WebElement folderSelect = wait.until(ExpectedConditions.presenceOfElementLocated(By.xpath("//div[@role='row' and .//a[contains(text(),'"+testFolder
folderSelect.click();
String optionsElementXpathString = "//div[@role='row' and .//a[contains(text(),'"+testFolder+"')]]//button[@aria-label='More Options']";
WebElement moreoptionsElement = wait.until(ExpectedConditions.presenceOfElementLocated(By.xpath(optionsElementXpathString)));
moreoptionsElement.click();
String actionsElementCssString = "li[class*=\"menu-item\"][class*=\"submenu-target\"]:not([class*=\"integrations-menu-item\"])";
WebElement moreActionsElement = wait.until(ExpectedConditions.presenceOfElementLocated(By.cssSelector(actionsElementCssString)));

Actions action = new Actions(driver);
action.moveToElement(moreActionsElement).build().perform();

String linkToRenameXpath = "//li[@class='menu-item submenu-target']//ul[@tabindex='-1']//span[contains(text(),'Rename')]";
WebElement renameFolder = wait.until(ExpectedConditions.presenceOfElementLocated(By.xpath(linkToRenameXpath)));
renameFolder.click();
```

Mouse Hover Actions Element

Drag And Drop Mouse Action

- **Actions** class has **dragAndDrop()** method which:
  - Performs click-and-hold at the location of the source element, moves to the location of the target element, then releases the mouse click
- **Actions dragAndDrop(WebElement source, WebElement target)** – Returns **Actions class** object. It takes two parameters:
  - **Source** is the element to emulate mouse button down
  - **Target** is the element to move to and release the mouse button




```
boolean bgcolor = false;
if(capitalName.equals("Oslo")) {
    if(countryList.contains("Norway")) {
        source = wait.until(ExpectedConditions
                .presenceOfElementLocated(By.xpath("//div[contains(@id,'box')][text()='Oslo']")));
        destination = wait.until(ExpectedConditions
                .presenceOfElementLocated(By.xpath("//div[contains(@id,'box')][text()='Norway']")));
        act.dragAndDrop(source, destination).build().perform();
        bgcolor = false;
        try {
            bgcolor = wait.until(ExpectedConditions.attributeContains(
                    By.xpath("//div[contains(@id,'box')][text()='Oslo']"), "style", "background-color: rgb(0, 255, 0)"));
        } catch (Exception e) {
            System.out.println("wrong drag");
        }
        if (bgcolor==true)
            System.out.println(capitalName +" Bgcolor is correct");
    }
}
```

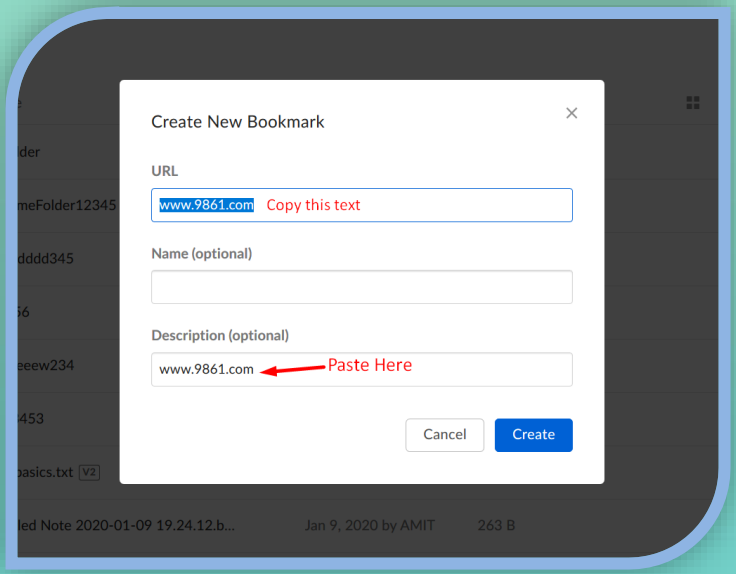
drag drop using Actions class

Also verifying colour changed or not after drag drop

Keyboard Simulation for Copy-Paste

- **Actions class** has **keyUp()** and **keyDown()** methods with which we can simulate pressing keyboard keys
- **Actions keyDown(CharSequence key)** - Performs a modifier key press and do not release it, so that subsequent interactions may assume it's already pressed
- **Actions keyUp(CharSequence key)** - Performs a modifier key release
- **Actions sendKeys(CharSequence... keys)** - Sends keys to the active element

Note: All these methods return **Actions class** object, making possible to call these methods in chain (**Action Chaining**) with **build().perform()** method




```
WebElement urlTextField = wait.until(ExpectedConditions.presenceOfElementLocated(By.cssSelector("input[data-resin-target=\"urlinput\"]")));

String bookmarkEntry = "www."+ (int)(Math.random()*((1000-1)+1))+1+".com";
urlTextField.sendKeys(bookmarkEntry);

Actions builder = new Actions(driver);
builder.keyDown( Keys.CONTROL ).sendKeys( "a" ).keyUp( Keys.CONTROL ).build().perform();

builder.keyDown( Keys.CONTROL ).sendKeys( "c" ).keyUp( Keys.CONTROL ).build().perform();
```

Simulating keyboard 'Ctrl+a' and ''Ctrl+c' in bookmark url field

```
WebElement descPaste = wait.until(ExpectedConditions.presenceOfElementLocated(By.cssSelector("input[data-resin-target=\"descriptioninput\"]")));
descPaste.click();
builder.keyDown( Keys.CONTROL ).sendKeys( "v" ).keyUp( Keys.CONTROL ).build().perform();
```

Simulating keyboard 'Ctrl+v' in bookmark description field

Keys class in WebDriver

- **org.openqa.selenium.Keys is a enum (group of named constants) representing keyboard pressable keys that aren't text.**
- Internally **enum** is the **implementation** of the **class** only.
- During **automation**, we are often required to press **enter**, **control**, **tab**, **arrow keys**, **function keys** and other **non-text keys** as well from the keyboard.

```
WebElement textbox = driver.findElement(By.id("idOfElement"));
textbox.sendKeys(Keys.ENTER);
```

- Using static method **chord()** of Keys:
  - **static String chord(CharSequence... value) -** Simulate pressing many keys at once in a "chord“
  - **Appends** each key value to a string, and adds the chord termination key (Keys.NULL) and **returns** the resultant **string**.

```
WebElement element2 = driver.findElement(By.cssSelector("li[class=\"gf-li\"]>a"));
String presscontrol = Keys.chord(Keys.CONTROL, Keys.RETURN);
element2.sendKeys(presscontrol);
```

```
public enum Keys implements CharSequence

  NULL          ('\uE000'),
  CANCEL        ('\uE001'), // ^break
  HELP          ('\uE002'),
  BACK_SPACE    ('\uE003'),
  TAB           ('\uE004'),
  CLEAR         ('\uE005'),
  RETURN        ('\uE006'),
  ENTER         ('\uE007'),
  SHIFT         ('\uE008'),
  LEFT_SHIFT    (Keys.SHIFT),
  CONTROL       ('\uE009'),
  LEFT_CONTROL  (Keys.CONTROL),
  ALT           ('\uE00A'),
  LEFT_ALT      (Keys.ALT),
  PAUSE         ('\uE00B'),
  ESCAPE        ('\uE00C'),
  SPACE         ('\uE00D'),
  PAGE_UP       ('\uE00E'),
  PAGE_DOWN     ('\uE00F'),
  END           ('\uE010'),
  HOME          ('\uE011'),
  LEFT          ('\uE012'),
  ARROW_LEFT    (Keys.LEFT),
  UP            ('\uE013'),
  ARROW_UP      (Keys.UP),
  RIGHT         ('\uE014'),
  ARROW_RIGHT   (Keys.RIGHT),
  DOWN          ('\uE015'),
  ARROW_DOWN    (Keys.DOWN),
  INSERT        ('\uE016'),
  DELETE        ('\uE017'),
  SEMICOLON     ('\uE018'),
  EQUALS        ('\uE019'),
```

Quiz

- What is the return type of **ExpectedConditions class, attribute validation** methods and **text present in a element** methods?
- Which two parameters are needed for **ExpectedConditions class**, **waiting for elements count** methods and what is their **return type**?
- Which class is used for simulating **Keyboard and Mouse actions** in **WebDriver**?
- What is **Action chaining**?
- How will you **compile** and then **execute** all the keyboard and mouse **user gestures**?
- Which method is used for **mouse hover** certain element?
- What is the purpose of **keyUp()** and **keyDown()** methods?
- What is the purpose of **Keys enum**?

Homework

- Complete Rename Folder WebDriver script:
  - **Assumption Folder** has been already renamed
  - Mouse Hover "**RenameFolder12345**" name link, so that pencil icon is visible
  - Click on the "**pencil icon"** link
  - Rename the folder back to "**RenameFolder**" in the textbox
  - Verify the "**Rename message**" notification and wait for it to disappear from screen
  - **Logout** from the portal

- Complete **Drag and Drop** script:
  - Navigate to http://dhtmlgoodies.com/scripts/drag-drop-custom/demo-drag-drop-3.html
  - **Drag And Drop** all **capital boxes** to their respective **country boxes**
  - Verify that the colour of **capital box** has changed to "**green**" after **dragging**